PROJECT OVERVIEW

HelpHero

Support desk platform

A full-stack application for creating, assigning, and resolving customer support tickets.

Ram Tichkule

Session 21 DevOps Final Capstone
Local Kubernetes demonstration, 16 October 2026

APPLICATION

Ticket workflow

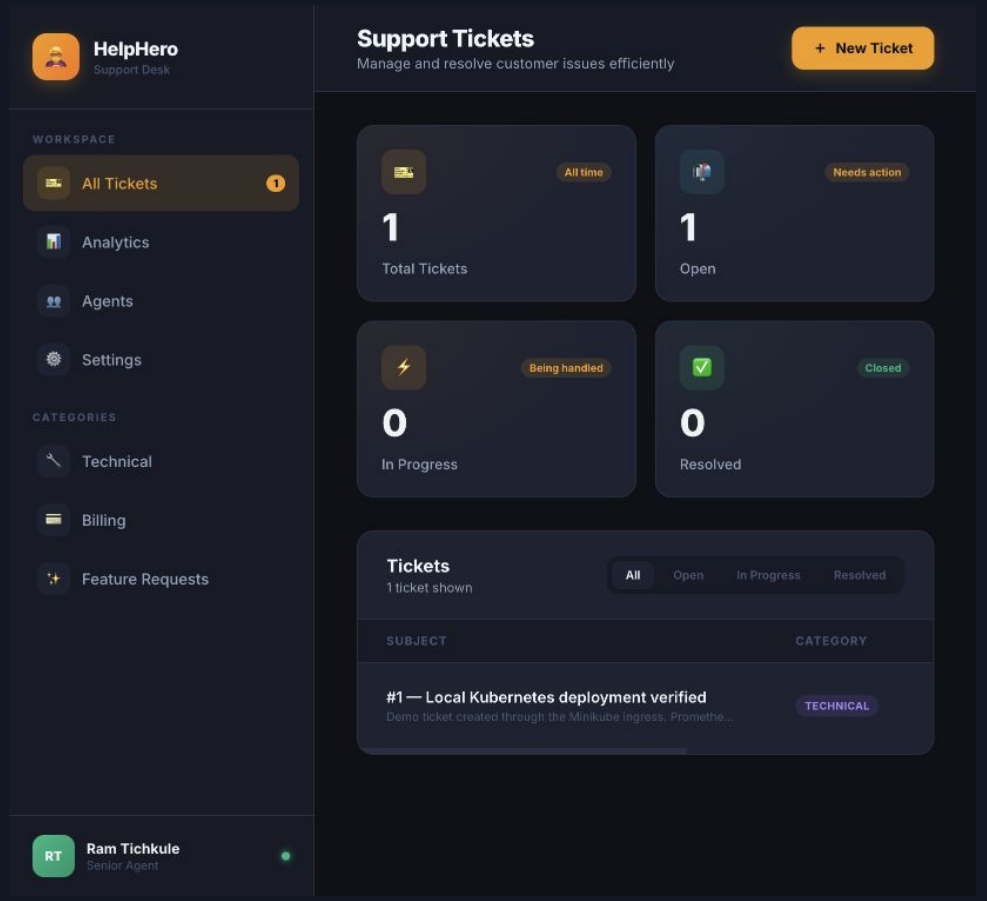
Create and triage

Subject, description, category, priority, and agent assignment.

Track resolution

Open, In Progress, and Resolved with counts in the dashboard.

Browser ticket creation succeeded through the local ingress.



IMPLEMENTATION

Technology stack

Application

React and Vite
FastAPI on Python 3.12

Data

PostgreSQL 16
SQLAlchemy and Alembic migrations

Delivery

Docker, GitHub Actions, GHCR
Trivy container scanning

Runtime

Terraform, Kubernetes, Helm
Prometheus and Grafana

ARCHITECTURE

Request and metrics paths

LOCAL REQUEST PATH

01 Browser and NGINX Ingress
02 / serves React through Nginx
03 /api routes to FastAPI
04 FastAPI reads and writes PostgreSQL

OBSERVABILITY PATH

Prometheus scrapes backend /metrics every 15 seconds.
Grafana queries Prometheus for dashboard panels.

AUTOMATION

CI/CD workflow

01 Test	Pytest and the Vite production build
02 Build	Backend and frontend Docker images
03 Scan	Trivy checks HIGH and CRITICAL findings
04 Publish	GHCR images tagged with commit SHA and latest
05 Deploy	Main pushes run a Kind backend smoke test

Configured workflow: tests, image publication, and a Kind smoke test.

AWS infrastructure configuration

Network design

ap-south-1, VPC 10.20.0.0/16
Two public and two private subnets
One NAT gateway

EKS design

Managed t3.medium node group
Desired size 2, maximum size 4
Private subnet placement

AWS demo workflow

Demo assumption: valid AWS credentials are configured.
Plan reviews changes. Apply provisions the VPC and EKS.
Destroy removes the managed infrastructure after the demo.

terraform.tfvars.example documents the deployment inputs.

LOCAL VERIFICATION

Kubernetes and Helm deployment

3 ready pods

Frontend, FastAPI backend, and PostgreSQL
Namespace: helphero
Helm release: helphero, revision 2

Deployment controls

Readiness and liveness probes
PostgreSQL persistent volume
Wait for PostgreSQL before startup

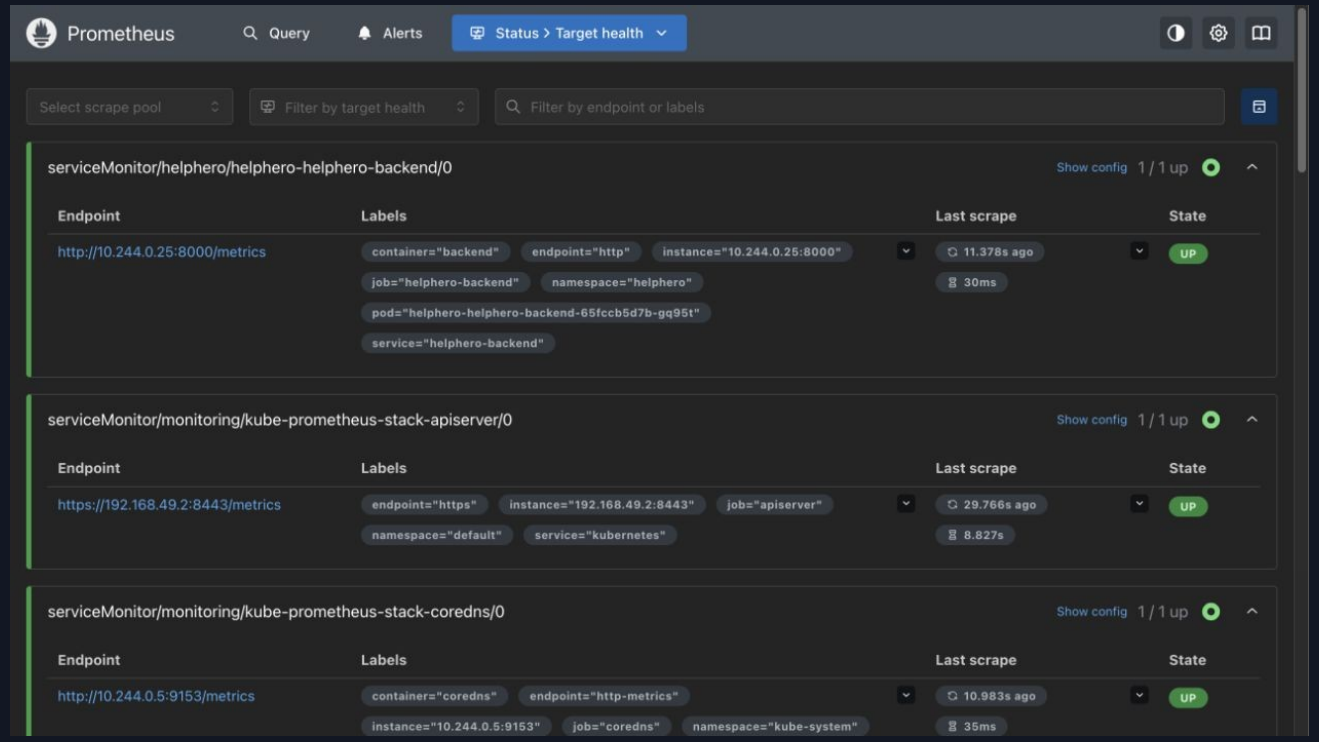
Ingress routing

helphero.127.0.0.1.nip.io:8080
Loopback forwarding to NGINX Ingress
Local HPA disabled

Minikube deployment verified. Cluster and access processes are now stopped.

Prometheus target verification

/metrics exposes request counters, latency histograms, and process metrics.

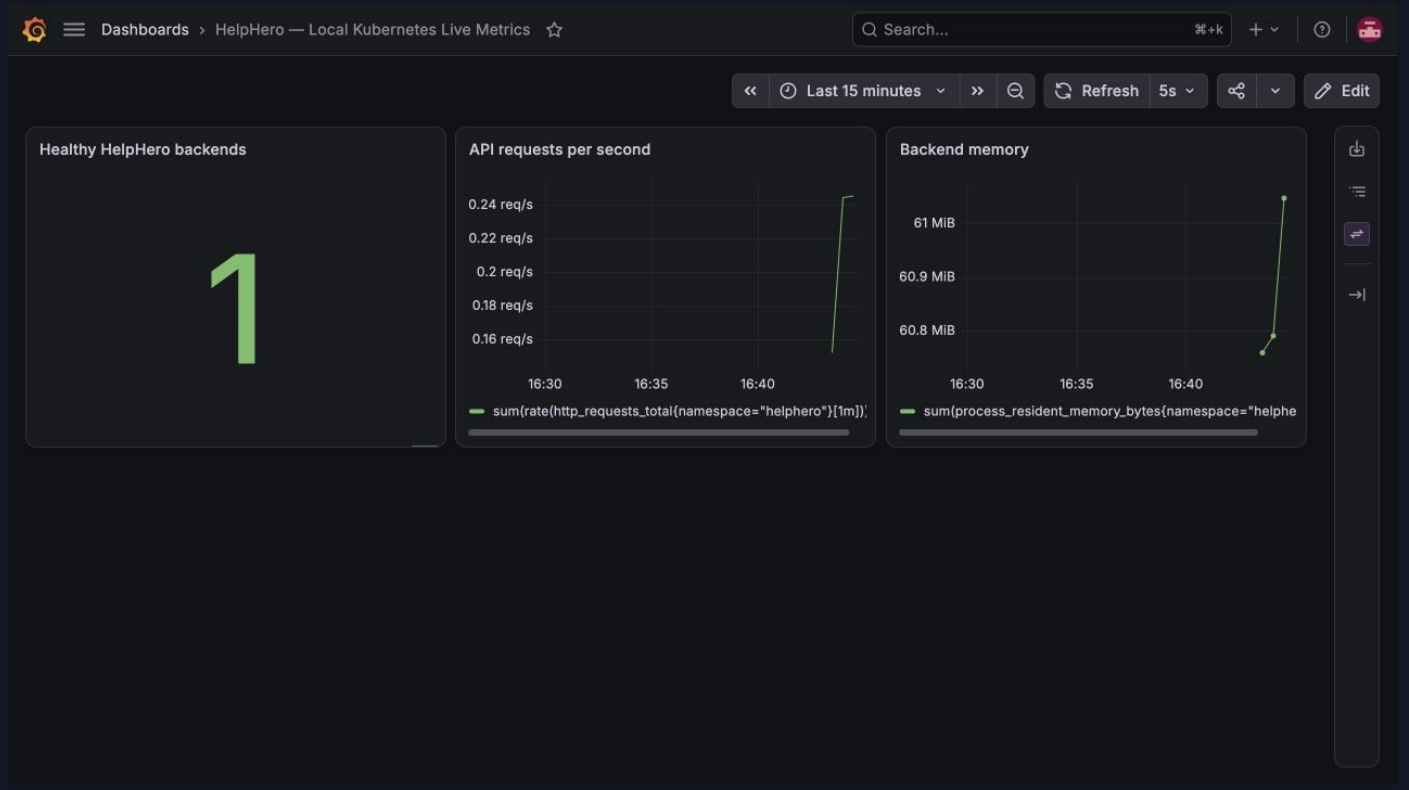


UP

HelpHero backend
15-second scrape interval
No scrape error

Grafana live dashboard

Three panels displayed real Prometheus data during the local run.



Learnings and next steps

What the local run proved

Ten API tests passed.
Ingress reached the API and database.
Prometheus and Grafana showed live data.

What needed fixing

Kubernetes service DNS differs from Compose.
Backend startup must wait for PostgreSQL.
Non-root tests need a writable database path.

Cloud deployment workflow

With valid AWS access, check supported versions.
Plan, apply, verify EKS, then destroy.

Delivery improvements

Pin dependencies and enforce scan failures.
Add frontend smoke tests and verify CI runs.
Review production secrets, TLS, and backups.